CARAT: Class-Aware Robust Aggregation via Hidden Task Certificates for Poisoning-Resilient Federated Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Federated learning servers must aggregate client updates without observing client data, which leaves training vulnerable to both untargeted model poisoning and targeted backdoor attacks. Existing robust aggregators mainly rely on update geometry, coordinate-wise robust statistics, or alignment with a single trusted reference direction, but these signals can be brittle when benign clients are heterogeneous and adaptive attackers stay near the benign update manifold. We present **CARAT**, a class-aware robust aggregation rule based on *hidden task certificates*: using server-side labeled reference data, CARAT instantiates multiple hidden, class-balanced micro-tasks and scores each client update by the loss reduction it induces across them. CARAT combines this task-aware signal with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization under a worst-task objective with temporal regularization, thereby favoring updates that remain broadly useful across hidden semantic slices while suppressing structurally extreme clients. In the strongest completed repository-backed CIFAR100/FangAttack pilot, CARAT suppresses attacker weights and substantially improves over Mean; however, because the current completed evidence is still training-only and the full clean-accuracy and ASR benchmark suite is not yet finished, these results should be read as initial empirical validation rather than a complete benchmark claim.

1 Introduction

Federated learning (FL) enables many clients to train a shared model without centralizing their raw data [1–3]. This decentralization is attractive in privacy- and governance-sensitive settings, but it also leaves the server in a fundamentally asymmetric position: it must update the global model using client messages that it cannot directly verify. Poisoning attacks exploit exactly this asymmetry. Malicious clients can slow convergence, distort the learned decision boundary, or implant triggers that activate attacker-chosen behavior at test time [4–11]. In practice, the robustness of FL is therefore tightly coupled to the aggregation rule.

The central difficulty is that malicious behavior and benign heterogeneity can look superficially similar. Under non-i.i.d. partitions, benign clients legitimately differ in direction, norm, sparsity, and class emphasis [12–15]. As a result, an update can appear unusual either because it is harmful or because it reflects a different local data distribution. Existing defenses often rely on signals that are too coarse to resolve this ambiguity reliably. Pairwise distances and geometric centrality can misclassify benign dispersion as adversarial deviation; coordinate-wise robust statistics suppress only marginal extremes and ignore cross-coordinate structure; and trusted-reference schemes reduce the

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

server's auxiliary information to a single global direction, which can miss failures concentrated on particular classes or semantic regions [16–23].

This leaves a key question: how should the server validate client updates when benign heterogeneity is intrinsic and the most harmful attacks may only reveal themselves on specific semantic slices of the task? What is needed is a validation signal that is semantically discriminative, robust to benign diversity, and difficult for attackers to satisfy by merely staying near the benign update manifold.

To address this, the paper proposes **CARAT**, *Class-Aware Robust Aggregation via Hidden Task Certificates*. CARAT assumes server-side access to labeled reference data and uses that data to instantiate hidden, class-balanced micro-tasks. Each client update is then scored by the loss reduction it induces across these tasks, producing a certificate matrix that measures whether the update is broadly useful across hidden semantic slices rather than merely geometrically central. CARAT combines this task-aware signal with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization under a worst-task objective with temporal regularization. Our main contributions are summarized as follows.

- We propose CARAT, a class-aware robust aggregation rule that validates client updates through hidden-task certificates built from server-side labeled reference data, replacing one-shot geometric trust signals with multi-task utility checks.
- We formulate aggregation as a capped-simplex optimization problem that combines worst-task utility across hidden tasks, coordinate-rank anomaly suppression, and temporal regularization, yielding a robust weighting mechanism that remains compatible with heterogeneous FL.
- We provide repository-backed empirical evidence for CARAT and clearly distinguish completed verified runs from the broader clean-accuracy and ASR benchmark suite that is still being finalized.

2 Related Work

Model poisoning attacks. Modern FL attacks are explicitly designed to remain effective under server-side robustness measures. Early work already showed that highly constrained adversaries can poison the global model while preserving substantial stealth, even when the server deploys Byzantine-style defenses [4–6]. On the untargeted side, MinMax, MinSum, ALIE, and related optimization-based attacks craft malicious updates that evade simple geometric or statistical filters while still degrading global learning [6, 7]. On the targeted side, model-replacement backdoors, distributed backdoor construction, trigger-tail attacks, and durable attacks such as Neurotoxin show that an update can appear globally plausible while implanting highly specific malicious behavior [8–11]. Recent surveys make the broader point explicit: once the server cannot inspect raw client data, poisoning and backdoor attacks become a first-class systems concern rather than an edge case [3, 38].

Defense and detection. Classical Byzantine-robust aggregation rules infer benignness from centrality. Krum and related geometry-based methods favor updates that remain close to other clients under pairwise distances [16, 17], while coordinate-wise median and trimmed mean suppress marginal extremes independently [18]. Later variants extend this family through suspicion scoring, regularized robust aggregation, coding-based redundancy, and geometric-median aggregation [19–22, 39]. These methods are appealing because they require limited server-side prior knowledge, but under heterogeneous data they may still confuse legitimate benign dispersion with malicious deviation. This limitation has motivated a second line of work that explicitly tackles heterogeneity, for example through FedProx- and SCAFFOLD-style stabilization, resampling, bucketing, local-SGD analysis, gradient splitting, nearest-neighbor mixing, label-skew-aware aggregation, and entropy-based public-data guidance [13–15, 24–30]. A third line augments aggregation with trusted data, history, or representation-level inspection. In collaborative-learning settings, AUROR already anticipated update screening for poisoning resistance [40]; in FL, FLTrust introduces a trusted server anchor, FoolsGold and FLDetector rely on historical similarity diagnostics, and FLARE, FLAME, DeepSight, CRFL, and FLIP use latent-space inspection, clustering, clipping, smoothing, or reverse-engineering-based backdoor mitigation [23, 31–37]. Recent work also revisits the cost of robustness itself, highlighting clipping design and the tradeoff between Byzantine resilience and no-attack accuracy [41, 42].

CARAT is closest to this broader defense-and-detection family, but differs in its validation principle: instead of trusting update similarity, one reference direction, or post-hoc clustering alone, it evaluates whether a client update remains useful across multiple hidden semantic sub-tasks.

3 Method

3.1 Formulation and Assumptions

We consider synchronous FL with n clients and a global parameter vector $w_r \in \mathbb{R}^d$ at round r. At each round, client i returns a message u_i^r, which the server maps to a vector-form update $\Delta_i^r \in \mathbb{R}^d$. For FedAvg, Δ_i^r can be read as the client model delta relative to the current global model; for gradient-style optimizers, it is the client update itself. The server must aggregate $\{\Delta_i^r\}_{i=1}^n$ into a single global step while a fraction ρ of clients may be malicious and may coordinate their behavior.

CARAT assumes the server has access to labeled reference data $\mathcal{D}^{\mathrm{ref}}$. From this data, it constructs a probe pool $\mathcal{P}$ and repeatedly samples hidden, class-balanced validation tasks. This is a stronger assumption than fully data-free robust aggregation, and it should be treated as part of the method definition rather than as an incidental implementation detail. The reason for making this assumption explicit is that CARAT uses the server's auxiliary data not to define one trusted direction, but to generate many hidden semantic checks against which candidate client updates can be evaluated.

The goal is to assign aggregation weights that preserve benign diversity while suppressing updates that are either semantically harmful or structurally anomalous. This distinction matters in non-i.i.d. FL: benign clients may differ substantially in direction, norm, and class emphasis, so geometric dispersion alone is not enough to identify attacks. CARAT therefore separates each round into three conceptual steps. It first places all client updates into a common evaluation geometry, then measures each client's task utility and structural abnormality, and finally converts these signals into robust aggregation weights. This separation is central to the method: evaluation should be comparable across clients, while the final server step should still remain conservative and robust.

3.2 Method Details

At round r, CARAT begins by converting all client messages into vector-form deltas and computing their norms. Before any semantic evaluation, the server robustly clips client updates to prevent a small number of large-magnitude messages from dominating the comparison stage. Let $\|\Delta_i\|_2$ denote the norm of client i's update. In our implementation, the clipping threshold τ_r is estimated by a MAD-based rule:

$$\tau_r = \mathrm{median}(\{\|\Delta_i\|_2\}) + k \cdot 1.4826 \cdot \mathrm{MAD}(\{\|\Delta_i\|_2\}), \tag{1}$$

and the clipped update is

$$\bar{\Delta}_i = \min\left(1, \frac{\tau_r}{\|\Delta_i\|_2 + \varepsilon}\right) \Delta_i. \tag{2}$$

CARAT then rescales each clipped update to a common evaluation radius q_r:

$$\tilde{\Delta}_i = q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2 + \varepsilon}, \tag{3}$$

where q_r is chosen as a quantile of the positive clipped norms. This normalization removes pure scale advantages during hidden-task comparison. Importantly, the normalized vectors are used only for evaluation; the final model update is still computed from the clipped vectors. In other words, CARAT uses normalization to create a fair scoring geometry, not to define the actual optimization step taken by the server.

After this preprocessing stage, CARAT samples T hidden tasks from the probe pool $\mathcal{P}$, with each task constructed to be approximately class-balanced. This is the source of the method's class-aware behavior. A malicious update can remain globally plausible while degrading only a subset of classes or semantic regions, whereas repeated balanced hidden tasks make these localized failures harder to hide. For each hidden task t, let $\ell_t(w)$ denote the average cross-entropy loss of model w on that task. CARAT defines a *hidden-task certificate*

$$c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i). \tag{4}$$

Here, "certificate" denotes an internal task-level utility score rather than a formal certified-robustness guarantee. Positive values indicate that client i improves task t relative to the current global model.

Stacking these scores yields a certificate matrix $C \in \mathbb{R}^{n \times T}$. Because different tasks may have different intrinsic difficulty and baseline loss scale, CARAT robustly standardizes each column of C before optimization. This preserves within-task ordering while making the task scores comparable across different hidden slices. The resulting matrix gives the server a much richer signal than client-to-client similarity alone: a client should be trusted because it remains broadly useful across hidden tasks, not merely because it lies near the geometric center of the client population.

Task utility alone is still insufficient, because a malicious client may overfit to the probe tasks or align with part of the semantic signal while remaining structurally extreme in parameter space. CARAT therefore augments hidden-task certificates with a coordinate-rank anomaly score. For a sampled coordinate set $S \subseteq \{1, \ldots, d\}$, it ranks clients coordinate-wise and measures the average normalized distance from the center rank:

$$r_i^{(S)} = \frac{1}{|S|} \sum_{j \in S} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|. \tag{5}$$

CARAT repeats this computation over several random coordinate subsets and aggregates the resulting values robustly. A client receives a large anomaly score if its update remains persistently extreme across many coordinates rather than only on a few isolated entries. After robust standardization, only positive deviations are retained as penalties. The signal is intentionally asymmetric: suspicious extremeness is penalized, whereas unusual centrality is not explicitly rewarded. Together, the certificate matrix and rank penalty separate two notions that are often conflated in robust aggregation: whether an update is useful, and whether it remains structurally atypical.

3.3 Weight Optimization and Aggregation

CARAT converts these signals into aggregation weights by solving a constrained optimization problem on the capped simplex

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\}, \tag{6}$$

where $\hat{\rho}$ is an estimate of the attacker fraction. The cap prevents any single client from dominating the aggregate and encodes a conservative trust budget at the weight level: even when a client appears useful, it cannot absorb arbitrary influence in one round.

Let $r \in \mathbb{R}^n$ denote the positive rank penalties and let α^{prev} be the previous round's weights. CARAT solves

$$\max_{\alpha \in \mathcal{A}(\hat{\rho})} \ \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\mathrm{prev}}\|_2^2, \tag{7}$$

where

$$\operatorname*{softmin}_{\beta}(s_1, \ldots, s_T) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right). \tag{8}$$

The first term is a smooth approximation to worst-task utility: large β places more emphasis on the weakest hidden tasks, while finite β preserves optimization stability. The second term penalizes structurally extreme clients, and the third term regularizes the solution toward the previous round to reduce unnecessary oscillation. Eq. (7) therefore encodes a clear decision rule: a client should retain influence only if it remains useful across hidden tasks and does not repeatedly appear as an outlier in rank space.

After optimization, CARAT aggregates the clipped updates:

$$\Delta_{\mathrm{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i. \tag{9}$$

This yields the final server step. The final update therefore inherits the robustness benefits of clipping while using weights learned from the richer evaluation geometry defined above. In practice, the main additional computation comes from evaluating n candidate updates over T hidden tasks and

computing rank penalties on coordinate subsamples; the probe-model forward passes dominate, while the ranking and optimization stages remain tunable through explicit budget parameters.

A compact round-level algorithm summary is provided in Appendix A. Formal properties of the normalization step, capped-simplex feasibility, softmin behavior, temporal regularization, and per-round complexity are deferred to Appendix B.

4 Experiments

4.1 Experimental Setup

Datasets and Implementation. All paper-side experiments currently use FedAvg on CIFAR100 with a ResNet-18 backbone. Keeping the dataset-model pair fixed across clean-reference, untargeted-poisoning, backdoor, and ablation runs ensures that observed changes can be attributed to the defense rather than to architecture changes. CIFAR100 is a useful stress test for CARAT because the method evaluates client updates on class-balanced hidden tasks, and a larger label space exposes class-selective failure modes more clearly than low-class-count benchmarks. CARAT is implemented as a server-side aggregator in FLPoison and operates directly on the vectorized client deltas produced by the runtime. Unless otherwise noted, all committed experiment specs use synchronous FedAvg with 20 clients, 300 communication rounds, local epoch count 4, batch size 64, and learning rate 0.05. For clean and untargeted runs, `metrics_exp*.csv` records per-round training and test accuracy and loss. For backdoor runs, we report clean test accuracy together with attack success rate (ASR), and attack-specific diagnostics are parsed from the detailed run logs.

Baselines and Default Setting. The clean-reference benchmark uses `NoAttack` and compares {Mean, FLTrust, MultiKrum, CARAT}. The untargeted poisoning benchmark evaluates {ALIE, MinMax, FangAttack} against the same defense set. The backdoor benchmark evaluates {BadNets, Neurotoxin} against {Mean, FLTrust, FLAME, DeepSight, CARAT}. We consider both i.i.d. partitions and Dirichlet non-i.i.d. splits: `clean_reference.yaml` and `main_untargeted.yaml` use $\alpha \in \{0.1, 0.5\}$, while `backdoor.yaml` uses i.i.d. plus $\alpha = 0.5$. The untargeted suite sweeps attacker fractions $\{0.2, 0.4\}$; the clean-reference and backdoor suites use 0 and 0.2, respectively. Each paper-side suite runs three repeats with effective seeds 42, 43, and 44. The default CARAT hyperparameters are: probe pool size 512, 8 hidden tasks, 4 probe samples per class, probe batch size 128, probe-radius quantile 0.5, MAD clipping with $k = 2.5$, default attacker prior $\hat{\rho} = 0.2$, certificate temperature $\beta = 12.0$, rank weight 0.20, prior weight 0.05, 4096 sampled coordinates, 5 rank subsamples, 60 optimization steps, and optimizer learning rate 0.5. One important caveat is that the strongest completed legacy CARAT pilot was produced from an earlier base config with `evaluate=false`. Its main-text evidence is therefore limited to training accuracy, training loss, and runtime, whereas all newly committed benchmark suites are configured to record test-time metrics.

4.2 Performance Evaluation

Performance of Defense Effectiveness under Various Attacks. The main comparative question is whether CARAT improves robustness under both untargeted and targeted poisoning. For untargeted attacks, Table 1 summarizes the strongest *completed CARAT evidence* currently verified end-to-end in the repository: a single-seed pilot FedAvg run on CIFAR100 with an i.i.d. split, 20 clients, 20% attackers (4 malicious client slots), ResNet-18, 300 rounds, and FangAttack. The available logs cover Mean, TriGuardFL, and CARAT under the same base training configuration. Because that pilot configuration sets `evaluate=false`, the table reports *training-only* diagnostics and runtime rather than clean-test accuracy or ASR.

CARAT substantially improves over simple averaging in the verified run. Relative to Mean, it increases round-299 training accuracy by about 5.9 points and reduces training loss by about 0.24. At the same time, the current evidence does *not* support a broader claim of uniform dominance across strong baselines: on this single training-only run, TriGuardFL finishes with slightly higher training accuracy, whereas CARAT attains the lowest training loss. We therefore interpret Table 1 as pilot evidence that the method is functioning as intended, not as a complete comparative benchmark.

An evaluation-enabled benchmark aligned with the protocol above is now running in the repository. In the current repository snapshot dated April 16, 2026, the first four completed tasks from

Defense	Train acc. @299	Train loss @299	Mean round time (s)	Total runtime
Mean	0.5699	1.5187	1.66	8m25s
TriGuardFL	**0.6385**	1.2932	7.37	37m07s
CARAT	0.6286	**1.2748**	9.49	47m37s

Table 1: Completed pilot evidence from repository logs for FedAvg on CIFAR100 under i.i.d. FangAttack. These are training-only diagnostics because that pilot configuration disables test-time evaluation.

`main_untargeted.yaml` all come from the i.i.d. ALIE, 20% attacker slice: three Mean repeats and one FLTrust repeat. The three Mean runs finish at epoch 299 with eval accuracy 57.55%, 57.97%, and 57.39% and eval loss 1.579, 1.544, and 1.554; the completed FLTrust run finishes at 47.25% eval accuracy and 1.937 eval loss. These logs confirm that the new benchmark path is recording test-time metrics as intended, but the corresponding CARAT tasks in this suite have not finished yet, so they do not yet support a CARAT comparison.

For targeted attacks, the final benchmark should report clean test accuracy and ASR side by side for {BadNets, Neurotoxin} against {Mean, FLTrust, FLAME, DeepSight, CARAT}, rather than collapsing untargeted robustness and backdoor resistance into a single claim. This separation matters because an update can be globally harmful, or harmful only on a trigger-defined slice.

The most diagnostic CARAT-specific signal available today is the behavior of the learned weights. In the verified FangAttack pilot, the coordinator assigns the first `num_adv` worker slots to attackers, so clients 0–3 are malicious. Late in training, CARAT repeatedly logs weight vectors of the form

$$\alpha = [0, 0, 0, 0, 0.0625, 0.0625, \ldots, 0.0625],$$

which means the optimizer places zero mass on the entire malicious coalition while distributing the remaining mass uniformly over benign clients. At epoch 298, for example, the log records task scores $[0.2879, 0.2784, 0.2859, 0.2781, 0.2533, 0.3009, 0.2930, 0.2876]$, a softmin value of 0.2820, evaluation radius 18.1484, and clip norm 18.3227. These quantities indicate that the implementation is doing more than hard filtering: it is identifying weak hidden tasks under a common scoring radius and then applying the optimized weights to the clipped updates used for aggregation.

This mechanistic evidence is encouraging, but it still does not establish improved clean-test accuracy, reduced ASR, or robustness across non-i.i.d. regimes for CARAT. The remaining empirical gap is no longer instrumentation but coverage: the evaluation-enabled CARAT benchmark has started, yet its CARAT rows, non-i.i.d. tasks, backdoor suite, and ablations are not complete.

Impact of Attacker Ratio. Attacker ratio is varied explicitly in the untargeted benchmark through `num_advs` $\in \{0.2, 0.4\}$. This sweep isolates robustness degradation as the malicious coalition grows while keeping the dataset, model, and optimization protocol fixed. The currently verified CARAT pilot corresponds to the 0.2 slice, whereas the 0.4 slice remains in the running evaluation-enabled suite. This axis is especially important for CARAT because the capped-simplex constraint and the default prior $\hat{\rho} = 0.2$ encode an explicit belief about how much malicious mass the aggregator should tolerate. Comparing 0.2 and 0.4 therefore tests whether the method remains stable when the true attack rate approaches or exceeds its default trust budget. The clean-reference suite uses attacker ratio 0 to measure the no-attack cost of robustness, and the backdoor suite fixes 0.2 so that clean accuracy and ASR can be compared without introducing another confound.

Compatibility with Cross-device and Cross-silo FL Scenarios. The current committed paper benchmarks are synchronous FedAvg runs with 20 clients, so they should be viewed as a controlled common-protocol setting rather than as a complete cross-device or cross-silo study. In practice, the present setup is closer to a small-scale cross-silo proxy than to a large-scale cross-device deployment. A more complete evaluation should therefore separate two deployment regimes. Cross-device-like settings should stress larger and more heterogeneous client populations, potentially with weaker per-client signal. Cross-silo-like settings should stress fewer but more stable participants whose updates carry stronger local information. FLPoison already supports multiple FL algorithms, datasets, models, and client-count configurations, so extending CARAT to both regimes is operationally feasible. However, the current manuscript should present this item as a compatibility target rather

Factor	Directly supported config keys
Probe construction	`num_probe_pool`, `num_probe_tasks`, `probe_samples_per_class`
Clipping / normalization	`clip_method`, `clip_k`, `clip_percentile`, `probe_radius_quantile`
Objective shaping	`certificate_beta`, `rank_weight`, `prior_weight`, `rho_hat`
Rank estimation	`rank_num_coords`, `rank_num_subsamples`
Optimizer budget	`optimizer_steps`, `optimizer_lr`

Table 2: Ablation axes already exposed by the current CARAT implementation. This is the safest starting point for a code-aligned ablation section.

than a finished empirical claim, since no dedicated cross-device or cross-silo CARAT benchmark is complete yet.

Aggregation Overhead. Aggregation overhead is a second practical criterion. Compared with lightweight rules such as Mean, CARAT adds repeated hidden-task forward passes, rank subsampling, and projected optimization. On the current FangAttack/CIFAR100 pilot, the mean round time is 9.49s for CARAT versus 1.66s for Mean and 7.37s for TriGuardFL. Any final empirical case must therefore justify the extra server-side cost with corresponding clean-test or ASR gains, not only with training-time diagnostics. This subsection focuses on server-side aggregation overhead rather than end-to-end system latency.

Impact of Key Hyperparameters. The cleanest hyperparameter study is the one already supported by the current configuration interface. The first ablation package should isolate:

- probe pool size,
- number of hidden tasks,
- per-class probe samples,
- clipping rule, clipping threshold, and evaluation-radius quantile,
- rank penalty weight λ_r,
- prior weight λ_p,
- softmin temperature β,
- rank subsampling budget and optimizer steps.

These ablations are directly supported by the preset groups summarized in Table 2. This design already supports a concrete methodological question: how much of CARAT's behavior comes from task construction, clipping and normalization, rank penalties, or temporal regularization?

Two additional structural ablations would strengthen the final paper but require small code patches rather than simple config changes: replacing the softmin objective with average utility, and bypassing hidden-task certificates entirely. The current implementation does not expose those switches as top-level parameters, so the paper should not imply that those ablations have already been run.

The intended empirical narrative is therefore sequential: establish clean and untargeted robustness first, evaluate backdoor robustness with both ASR and clean utility second, and then use the supported ablations above to determine whether CARAT's gains come primarily from hidden-task validation, rank-space filtering, clipping and common-radius evaluation, or their interaction.

A core methodological limitation remains the server-side data assumption already stated in the Method section: in the current code path, the probe pool is sampled from a server-visible labeled training dataset rather than from a tiny external reference set. This assumption matters when CARAT is compared to fully data-free robust aggregators. Empirical coverage is also still incomplete. The strongest completed CARAT evidence is a single-seed, training-only FangAttack pilot. An evaluation-enabled benchmark suite that includes CARAT is now committed and running, but the CARAT rows, non-i.i.d. cases, backdoor tasks, and adaptive-attack studies are not yet complete. This means the current manuscript supports a methodology claim and an initial empirical result, but not yet a full benchmark claim.

5 Conclusion

We presented CARAT, a class-aware robust aggregation rule that validates client updates through hidden task certificates rather than relying only on geometric centrality or a single trusted reference direction. By scoring updates across multiple hidden, class-balanced semantic slices of the task and combining those scores with rank-space anomaly suppression and capped-simplex weight optimization, CARAT is designed to preserve benign heterogeneity while reducing the influence of semantically harmful or structurally extreme clients.

The current repository-backed evidence supports this mechanism claim at a preliminary empirical level: in the completed CIFAR100/FangAttack pilot, CARAT suppresses attacker weights and substantially improves over Mean. At the same time, the present manuscript does not yet support a full benchmark claim, because the clean-accuracy, ASR, non-i.i.d., and multi-seed CARAT suites are still being completed, and the method retains the explicit assumption of server-side labeled reference data. Even with these limitations, the hidden-task certificate view provides a concrete and technically grounded direction for robust FL aggregation, especially in settings where class-selective failures matter.

References

[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 2017.

[2] Kallista A. Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[3] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurelien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. *Advances and Open Problems in Federated Learning*. Now Publishers, 2021.

[4] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *International Conference on Machine Learning*, pages 634–643, 2019.

[5] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to Byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.

[6] Virat Shejwalkar and Amir Houmansadr. Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *Network and Distributed System Security Symposium*, 2021.

[7] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. In *Advances in Neural Information Processing Systems*, 2019.

[8] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *International Conference on Artificial Intelligence and Statistics*, pages 2938–2948, 2020.

[9] Chulin Xie, Keli Huang, Pin-Yu Chen, and Bo Li. DBA: Distributed backdoor attacks against federated learning. In *International Conference on Learning Representations*, 2020.

[10] Hongyi Wang, Kartik Sreenivasan, Shashank Rajput, Harit Vishwakarma, Saurabh Agarwal, Jy-yong Sohn, Kangwook Lee, and Dimitris Papailiopoulos. Attack of the tails: Yes, you really can backdoor federated learning. In *Advances in Neural Information Processing Systems*, 2020.

[11] Zhengming Zhang, Ashwinee Panda, Linyue Song, Yaoqing Yang, Michael Mahoney, Prateek Mittal, Ramchandran Kannan, and Joseph Gonzalez. Neurotoxin: Durable backdoors in federated learning. In *International Conference on Machine Learning*, pages 26429–26446, 2022.

[12] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, et al. Federated learning with non-IID data. *arXiv preprint arXiv:1806.00582*, 2018.

[13] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, 2020.

[14] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning*, pages 5132–5143, 2020.

[15] Hangyu Zhu, Jinjin Xu, Shiqing Liu, and Yaochu Jin. Federated learning on non-IID data: A survey. *Neurocomputing*, 465:371–390, 2021.

[16] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, 2017.

[17] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *International Conference on Machine Learning*, 2018.

[18] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, 2018.

[19] Cong Xie, Sanmi Koyejo, and Indranil Gupta. Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance. In *International Conference on Machine Learning*, pages 6893–6901, 2019.

[20] Lingjiao Chen, Hongyi Wang, Zachary Charles, and Dimitris Papailiopoulos. DRACO: Byzantine-resilient distributed training via redundant gradients. In *International Conference on Machine Learning*, pages 903–912, 2018.

[21] Liping Li, Wei Xu, Tianyi Chen, Georgios B. Giannakis, and Qing Ling. RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets. In *AAAI Conference on Artificial Intelligence*, pages 1544–1551, 2019.

[22] Krishna Pillutla, Sham M. Kakade, and Zaid Harchaoui. Robust aggregation for federated learning. *IEEE Transactions on Signal Processing*, 70:1142–1154, 2022.

[23] Xiaoyu Cao, Minghong Fang, Jia Liu, Neil Zhenqiang Gong, and Yixin Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021.

[24] Lie He, Sai Praneeth Karimireddy, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via resampling. In *International Conference on Learning Representations*, 2021.

[25] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via bucketing. In *International Conference on Learning Representations*, 2022.

[26] Deepesh Data and Suhas Diggavi. Byzantine-resilient high-dimensional SGD with local iterations on heterogeneous data. In *International Conference on Machine Learning*, pages 2478–2488, 2021.

[27] Yuchen Liu, Chen Chen, Lingjuan Lyu, Fangzhao Wu, Sai Wu, and Gang Chen. Byzantine-robust learning on heterogeneous data via gradient splitting. In *International Conference on Machine Learning*, pages 21404–21425, 2023.

[28] Youssef Allouah, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Rafael Pinot, and John Stephan. Fixing by mixing: A recipe for optimal byzantine ML under heterogeneity. In *International Conference on Artificial Intelligence and Statistics*, pages 1232–1300, 2023.

[29] Wenxuan Bao, Jun Wu, and Jingrui He. BOBA: Byzantine-robust federated learning with label skewness. In *International Conference on Artificial Intelligence and Statistics*, pages 892–900, 2024.

[30] Wenke Huang, Zekun Shi, Mang Ye, He Li, and Bo Du. Self-driven entropy aggregation for byzantine-robust heterogeneous federated learning. In *International Conference on Machine Learning*, pages 20096–20110, 2024.

[31] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in sybil settings. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2020.

[32] Zaixi Zhang, Xiaoyu Xu, Jinyuan Jia, Matthew Jagielski, Gauri Joshi, Mingyi Hong, and Carl A. Gunter. FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[33] Ning Wang, Yang Xiao, Yimin Chen, Yang Hu, Wenjing Lou, and Y. Thomas Hou. FLARE: Defending federated learning against model poisoning attacks via latent space representations. In *ACM Asia Conference on Computer and Communications Security*, pages 946–958, 2022.

[34] Thien Duc Nguyen, Phillip Rieger, Hieu Yalame, Hauke Möllering, Huili Chen Xu, Ryohei Myung, Swarun Kadhe, and Kang G. Shin. FLAME: Taming backdoors in federated learning. In *USENIX Security Symposium*, 2022.

[35] Phillip Rieger, Thien Duc Nguyen, Hauke Möllering, Hieu Yalame, Huili Chen Xu, Ryohei Myung, Swarun Kadhe, and Kang G. Shin. DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection. In *Network and Distributed System Security Symposium*, 2022.

[36] Chulin Xie, Minghao Chen, Pin-Yu Chen, and Bo Li. CRFL: Certifiably robust federated learning against backdoor attacks. In *International Conference on Machine Learning*, pages 11372–11382, 2021.

[37] Kaiyuan Zhang, Guanhong Tao, Qiuling Xu, Siyuan Cheng, Shengwei An, Yingqi Liu, Shiwei Feng, Guangyu Shen, Pin-Yu Chen, Shiqing Ma, and Xiangyu Zhang. FLIP: A provable defense framework for backdoor mitigation in federated learning. In *International Conference on Learning Representations*, 2023.

[38] Pengrui Liu, Xiangrui Xu, and Wei Wang. Threats, attacks and defenses to federated learning: Issues, taxonomy and perspectives. *Cybersecurity*, 5(4), 2022.

[39] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Defending against saddle point attack in byzantine-robust distributed learning. In *International Conference on Machine Learning*, pages 7074–7084, 2019.

[40] Shiqi Shen, Shruti Tople, and Prateek Saxena. AUROR: Defending against poisoning attacks in collaborative deep learning systems. In *Annual Computer Security Applications Conference*, 2016.

[41] Youssef Allouah, Rachid Guerraoui, Nirupam Gupta, Ahmed Jellouli, Geovani Rizk, and John Stephan. Adaptive gradient clipping for robust federated learning. In *International Conference on Learning Representations*, 2025.

[42] Yi-Rui Yang, Chang-Wei Shi, and Wu-Jun Li. On the tension between byzantine robustness and no-attack accuracy in distributed learning. In *International Conference on Machine Learning*, 2025.

A Algorithmic Details

A.1 Round-Level Pseudocode

Algorithm 1 gives a standard round-level pseudocode view of one CARAT server round.

Algorithm 1 CARAT server aggregation at round r

Require: Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, probe pool $\mathcal{P}$, CARAT hyperparameters
Ensure: Aggregated update Δ_{agg}, updated temporal prior α^{prev}

```
if P is uninitialized then
    Build a balanced probe pool from the server-visible training dataset
end if
if r = 1 then
    α^prev ← (1/n) 1
end if
for i = 1 to n do
    Convert u_i^r into a vector-form delta Δ_i^r
end for
Compute the clipping threshold τ_r from {‖Δ_i^r‖_2}_{i=1}^n
for i = 1 to n do
    Δ̄_i^r ← min(1, τ_r / (‖Δ_i^r‖_2 + ε)) Δ_i^r
end for
Choose the evaluation radius q_r from a quantile of positive clipped norms
for i = 1 to n do
    Δ̃_i^r ← q_r Δ̄_i^r / (‖Δ̄_i^r‖_2 + ε)
end for
Sample T class-balanced hidden tasks from P with replacement/subset fallback if strict balanced
sampling fails
for t = 1 to T do
    Evaluate the baseline loss ℓ_t(w_r)
end for
for i = 1 to n do
    for t = 1 to T do
        c_{i,t} ← ℓ_t(w_r) − ℓ_t(w_r + Δ̃_i^r)
    end for
end for
Robustly standardize certificate columns to obtain C^std
Compute rank penalties r from repeated coordinate subsamples of {Δ̄_i^r}_{i=1}^n
α^(0) ← α^prev
for k = 0 to K − 1 do
    z ← (C^std)^⊤ α^(k)
    π ← softmin task weights induced by z and β
    g ← C^std π − λ_r r − 2λ_p (α^(k) − α^prev)
    α^(k+1) ← Π_{A(ρ̂)}(α^(k) + ηg)
end for
α* ← α^(K) and α^prev ← α*
Δ_agg ← Σ_{i=1}^n α_i* Δ̄_i^r
return Δ_agg, α^prev
```

A.2 Implementation Notes

Inputs. Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, server-visible training dataset, and CARAT hyperparameters.

Persistent state. CARAT maintains two pieces of state across rounds: a balanced probe pool sampled at initialization from the server-visible training dataset, and the previous-round weight vector α^{prev}, which serves as the temporal prior in the current round.

Initialization. At the first round, the implementation initializes α^{prev} to the uniform vector. This makes the initial round unbiased across clients while still allowing later rounds to exploit temporal consistency once CARAT has observed several optimization trajectories.

Reference-pool construction and hidden-task sampling. Let $\mathcal{P}_c \subseteq \mathcal{P}$ denote the subset of probe examples with class label c. A hidden task is constructed by sampling a small number of examples from each available class pool and concatenating them into one balanced micro-task. If the desired number of samples per class is not available without replacement, CARAT falls back to replacement and, if needed, to a random subset of the probe pool rather than aborting the round. This fail-soft design preserves the intended semantic diversity of the task sampler while ensuring that the server always obtains a well-defined certificate matrix.

Robust score calibration. For notation economy, the main text reuses C after standardization. In implementation terms, the raw certificate matrix is first transformed column-wise into

$$C^{\mathrm{std}}_{i,t} = \frac{c_{i,t} - \mathrm{median}_j(c_{j,t})}{1.4826 \cdot \mathrm{MAD}_j(c_{j,t}) + \varepsilon}.$$

This removes task-specific scale effects while preserving within-task ordering by Proposition B.4. Rank penalties are calibrated similarly. For each coordinate subsample S_s, let

$$\rho_i^{(s)} = \frac{1}{|S_s|} \sum_{j \in S_s} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|.$$

CARAT then computes a standardized rank score

$$z_i^{(s)} = \frac{\rho_i^{(s)} - \mathrm{median}_\ell(\rho_\ell^{(s)})}{1.4826 \cdot \mathrm{MAD}_\ell(\rho_\ell^{(s)}) + \varepsilon},$$

and aggregates subsamples through

$$r_i = \max\left(0, \mathrm{median}_s\, z_i^{(s)}\right).$$

As a result, a client is penalized only when it is consistently extreme across repeated coordinate views, not because of a single noisy subsample.

Projected optimization details. Again writing C for the standardized certificate matrix, CARAT initializes the optimizer with $\alpha^{(0)} = \alpha^{\mathrm{prev}}$ after the first round, and with the uniform vector at initialization. If

$$z(\alpha) = C^\top \alpha, \qquad \pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then the ascent direction used in projected updates is

$$g(\alpha) = C\pi(\alpha) - \lambda_r r - 2\lambda_p(\alpha - \alpha^{\mathrm{prev}}).$$

With optimizer step size η, one iteration takes the form

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})}\left(\alpha^{(k)} + \eta g(\alpha^{(k)})\right).$$

The projection maintains feasibility throughout optimization, so every intermediate iterate already obeys the nonnegativity, unit-sum, and per-client cap constraints described in the main text.

Round invariants and failure handling. Two implementation choices are important for robustness. First, the optimizer never leaves the capped simplex, so every intermediate and final weight vector remains feasible. Second, probe-task construction is fail-soft rather than fail-stop: if strict class-balanced sampling is impossible, the code falls back to replacement or to a random subset, ensuring that the round can still proceed with a well-defined certificate matrix.

Outputs. Aggregated server update, updated temporal prior α^{prev}, and diagnostic logs including learned weights, task scores, softmin focus, rank penalties, evaluation radius, clipping threshold, and the identities of the hidden probe tasks used in the current round.

Practical diagnostics. The most useful round-level diagnostics for debugging and empirical interpretation are the learned weights, the weakest-task values emphasized by the softmin, the clip threshold τ_r, the evaluation radius q_r, and the final rank penalties r_i. In successful runs, these quantities help distinguish three qualitatively different cases: benign but diverse clients that retain moderate weight, clearly harmful clients that lose certificate support across multiple tasks, and structurally extreme clients whose influence is reduced primarily by the rank penalty.

B Theoretical Analysis

This appendix records the main structural properties used to interpret CARAT. These results are not presented as a complete robustness guarantee against arbitrary adaptive attackers. Instead, they clarify why the method decomposes naturally into a normalized evaluation geometry, a task-aware utility objective, and a constrained robust aggregation step.

Proposition B.1 (Feasibility and support of the capped simplex)**.** *Let*

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\},$$

with $\hat{\rho} < 1$. Then $\mathcal{A}(\hat{\rho})$ is non-empty, and every $\alpha \in \mathcal{A}(\hat{\rho})$ satisfies

$$|\operatorname{supp}(\alpha)| \geq (1-\hat{\rho})n.$$

Proof sketch. Let $u = 1/((1-\hat{\rho})n)$. Since $\hat{\rho} < 1$, we have $u \geq 1/n$, so the uniform weight vector is feasible and the set is non-empty. For the support bound, if only k coordinates of α are positive, then

$$1 = \sum_{i=1}^{n} \alpha_i \leq ku = \frac{k}{(1-\hat{\rho})n},$$

which implies $k \geq (1-\hat{\rho})n$. Therefore CARAT can suppress suspicious clients, but cannot collapse the aggregate onto an arbitrarily small subset of participants. □

Proposition B.2 (Softmin bounds and gradient form)**.** *Let $s \in \mathbb{R}^T$ and define*

$$\operatorname*{softmin}_{\beta}(s) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right).$$

Then

$$\min_t s_t \leq \operatorname*{softmin}_{\beta}(s) \leq \min_t s_t + \frac{\log T}{\beta}.$$

Moreover,

$$\lim_{\beta \to \infty} \operatorname*{softmin}_{\beta}(s) = \min_t s_t, \qquad \lim_{\beta \to 0} \operatorname*{softmin}_{\beta}(s) = \frac{1}{T} \sum_{t=1}^{T} s_t.$$

Moreover, if $z(\alpha) = C^\top \alpha$ and

$$\pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then

$$\nabla_\alpha \operatorname*{softmin}_{\beta}(C^\top \alpha) = C\pi(\alpha).$$

Proof sketch. Let $m = \min_t s_t$. Since

$$e^{-\beta m}/T \leq \frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \leq e^{-\beta m},$$

applying $-(1/\beta)\log(\cdot)$ yields

$$m \leq \operatorname*{softmin}_{\beta}(s) \leq m + \frac{\log T}{\beta}.$$

The limit $\beta \to \infty$ follows immediately from the vanishing gap bound. For $\beta \to 0$, a first-order expansion of the exponential average gives

$$\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} = 1 - \beta \frac{1}{T} \sum_{t=1}^{T} s_t + o(\beta),$$

so the softmin converges to the arithmetic mean. Differentiating the softmin expression with respect to z_t yields the Gibbs weights $\pi_t(\alpha)$, and the chain rule then gives $C\pi(\alpha)$. Hence the ascent direction is a convex combination of certificate columns, with more weight placed on tasks whose current aggregate utility is smaller. □

Proposition B.3 (Directional invariance of certificate scoring)**.** *Suppose two nonzero clipped updates satisfy $\bar{\Delta}_i = a\bar{\Delta}_j$ for some scalar $a > 0$. Then their normalized versions in Eq.* (3) *coincide, i.e.,*

$$\tilde{\Delta}_i = \tilde{\Delta}_j.$$

Consequently, their hidden-task certificates are identical:

$$c_{i,t} = c_{j,t} \qquad \textit{for all } t \in \{1, \dots, T\}.$$

Proof sketch. Eq. (3) rescales every nonzero clipped update to the common radius q_r. Positive collinearity is therefore removed by normalization:

$$q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2} = q_r \frac{a\bar{\Delta}_j}{\|a\bar{\Delta}_j\|_2} = q_r \frac{\bar{\Delta}_j}{\|\bar{\Delta}_j\|_2}.$$

Substituting the resulting equality into Eq. (4) gives identical task scores. Thus, after clipping, the certificate stage depends on direction rather than residual norm magnitude. □

Proposition B.4 (Task-wise standardization preserves within-task ordering)**.** *Fix a task t and define a standardized certificate score*

$$\hat{c}_{i,t} = \frac{c_{i,t} - m_t}{s_t + \varepsilon},$$

where $m_t \in \mathbb{R}$ and $s_t > 0$ depend only on task t and not on client i. Then for any clients i, j,

$$c_{i,t} \geq c_{j,t} \quad \Longleftrightarrow \quad \hat{c}_{i,t} \geq \hat{c}_{j,t}.$$

Moreover,

$$\hat{c}_{i,t} - \hat{c}_{j,t} = \frac{c_{i,t} - c_{j,t}}{s_t + \varepsilon}.$$

Proof sketch. For a fixed task t, the map $x \mapsto (x - m_t)/(s_t + \varepsilon)$ is affine with strictly positive slope because $s_t + \varepsilon > 0$. Hence it preserves ordering. The difference identity follows by direct subtraction. Therefore column-wise robust standardization calibrates cross-task scale without changing the within-task ranking induced by the raw certificate scores. □

Proposition B.5 (Temporal prior and projected-iterate invariants)**.** *Define*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2.$$

If $\alpha, \alpha' \in \mathcal{A}(\hat{\rho})$ satisfy $C^\top \alpha = C^\top \alpha'$ and $r^\top \alpha = r^\top \alpha'$, then

$$f(\alpha) - f(\alpha') = -\lambda_p \left(\|\alpha - \alpha^{\text{prev}}\|_2^2 - \|\alpha' - \alpha^{\text{prev}}\|_2^2 \right).$$

In addition, under projected gradient ascent

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})} \left(\alpha^{(k)} + \eta \nabla f(\alpha^{(k)}) \right),$$

every iterate remains in $\mathcal{A}(\hat{\rho})$.

Proof sketch. The first statement follows by direct substitution: if the certificate and rank terms match, the only remaining difference in objective value is the quadratic prior term. CARAT therefore prefers, among utility-equivalent feasible weights, the solution closer to α^{prev}. The second statement is immediate from the projection operator. Hence the optimizer preserves three round invariants throughout: $\alpha_i \geq 0$, $\sum_i \alpha_i = 1$, and $\alpha_i \leq 1/((1 - \hat{\rho})n)$. □

Proposition B.6 (Norm bound for the final aggregate)**.** *Suppose each clipped update satisfies $\|\bar{\Delta}_i\|_2 \leq \tau_r$ and $\alpha \in \mathcal{A}(\hat{\rho})$. Then the final aggregate*

$$\Delta_{\text{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i$$

satisfies

$$\|\Delta_{\text{agg}}\|_2 \leq \tau_r.$$

Proof sketch. Because $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$, the aggregate is a convex combination of clipped updates. By the triangle inequality,

$$\|\Delta_{\text{agg}}\|_2 \leq \sum_{i=1}^{n} \alpha_i \|\bar{\Delta}_i\|_2 \leq \sum_{i=1}^{n} \alpha_i \tau_r = \tau_r.$$

Thus the final server step inherits the clipping radius as a hard norm upper bound. □

Proposition B.7 (Per-round complexity)**.** *Let n be the number of clients, T the number of hidden tasks, m the number of sampled coordinates for rank scoring, S the number of rank subsamples, and K the number of projected-gradient steps. Ignoring constant factors from batch size and model width, the extra defense-side work of CARAT scales as*

$$\textit{probe-model forward passes} + O(Snm + KnT).$$

Proof sketch. CARAT performs one baseline probe evaluation plus n candidate probe evaluations over the T hidden tasks, then computes S rank summaries over m coordinates, and finally executes K projected optimization steps over n weights and T task scores. In the current implementation, the probe-model forward passes dominate wall-clock time, while the rank and optimization stages are lower-order terms controlled by m, S, and K. □

Remark on assumptions. The theoretical object D^{ref} in the main text should be read as the probe pool *after* subsampling. The code, however, obtains that pool by sampling from the full server-visible training dataset. Any final formal discussion should keep this distinction explicit, because it materially strengthens the server-side assumption relative to a fully data-free defense.

C Additional Experimental Details

This section summarizes the current verified experimental scope together with the next empirical extensions already supported by the repository structure.

Currently verified run. The strongest directly verified completed CARAT run in the repository uses FedAvg, CIFAR100, ResNet-18, an i.i.d. partition, 20 clients, 4 attackers, FangAttack, 300 rounds, learning rate 0.05, local epochs 4, and `evaluate=false`. The main text reports the final training metrics and runtime of this run together with the corresponding Mean and TriGuardFL baselines. In the current repository snapshot dated April 16, 2026, the evaluation-enabled `main_untargeted.yaml` sweep has also started producing ALIE baseline logs, but no CARAT task from that sweep is complete yet.

Supported benchmark matrix. The committed `main_untargeted.yaml` spec already expands to 216 tasks over i.i.d. and non-i.i.d. splits with Dirichlet $\alpha \in \{0.1, 0.5\}$, attacker fractions $\{0.2, 0.4\}$, untargeted attacks {ALIE, MinMax, FangAttack}, defenses {Mean, FLTrust, MultiKrum, CARAT}, and three repeats. Companion specs `clean_reference.yaml`, `backdoor.yaml`, and `ablations.yaml` cover clean baselines, backdoor evaluation, and component analysis.

Next additions.

1. Complete the running `main_untargeted.yaml` sweep, prioritizing the CARAT rows and reporting multi-seed means and variances for clean test accuracy and loss.
2. Extend the verified runs from the current i.i.d. slice to the non-i.i.d. Dirichlet settings already present in `main_untargeted.yaml`.
3. Complete the committed `backdoor.yaml` suite so that the paper reports both clean utility and ASR.
4. Run the config-supported ablations in Table 2, prioritizing the number of hidden tasks, the rank and prior weights, and the clipping controls.
5. Report multi-seed means and variances, plus defense runtime overhead, before making any strong comparative claim against other robust aggregators.